

Preliminary Task 3: Neural Network vs Logistic PCA

Anishkumar Senthil

```
import pyreadr
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, f1_score

from tensorflow import keras

RDATA_PATH = "../data/claims-clean-example.RData"

result = pyreadr.read_r(RDATA_PATH)

if "claims_clean" in result:
    df = result["claims_clean"]
else:
    df = next(iter(result.values()))
```

WARNING:tensorflow:From C:\ANISH-OLDLENOVO\Anish_School\Pstat197\tfenv\lib\site-packages\keras\src\losses.py:2976: The name tf.losses.sparse_softmax_cross_entropy is deprecated

```
cols = list(df.columns)
print("Columns in data:", cols)

id_col = None
```

```

text_col = None
label_col = None

if ".id" in cols:
    id_col = ".id"
else:
    for c in cols:
        if "id" in c.lower():
            id_col = c
            break

for c in cols:
    if "text_tmp" in c.lower():
        text_col = c
        break
if text_col is None:
    for c in cols:
        if "text" in c.lower():
            text_col = c
            break

if "bclass" in cols:
    label_col = "bclass"

print("Detected id_col:", id_col)
print("Detected text_col:", text_col)
print("Detected label_col:", label_col)

if id_col is None or text_col is None or label_col is None:
    raise ValueError(
        "Could not find id/text/label columns. "
        "Check 'Columns in data:' and set id_col/text_col/label_col manually."
    )

df = df[[id_col, text_col, label_col]].dropna()
df = df.rename(columns={id_col: ".id", text_col: "text_tmp", label_col: "bclass"})
df["text_tmp"] = df["text_tmp"].astype(str)

df["bclass"] = df["bclass"].astype("category")
class_names = list(df["bclass"].cat.categories)
y = df["bclass"].cat.codes.to_numpy()
X_text = df["text_tmp"].tolist()

```

```

print("Number of rows:", len(df))
print("Class labels:", class_names)
print("Label counts:", np.bincount(y))

```

```

X_train_text, X_test_text, y_train, y_test = train_test_split(
    X_text,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)

tfidf = TfidfVectorizer(
    max_features=20000,
    ngram_range=(1, 2),
    stop_words="english",
)

X_train_tfidf = tfidf.fit_transform(X_train_text)
X_test_tfidf = tfidf.transform(X_test_text)

print("TF-IDF shape (train):", X_train_tfidf.shape)
print("TF-IDF shape (test):", X_test_tfidf.shape)

n_components = 100

logit_pca_model = Pipeline([
    ("svd", TruncatedSVD(n_components=n_components, random_state=42)),
    ("clf", LogisticRegression(max_iter=1000)),
])

print("\nFitting logistic PCA baseline")
logit_pca_model.fit(X_train_tfidf, y_train)
logit_pred = logit_pca_model.predict(X_test_tfidf)

logit_acc = accuracy_score(y_test, logit_pred)
logit_f1 = f1_score(y_test, logit_pred, average="binary")

print("=== Logistic PCA baseline ===")

```

```

print(f"Accuracy: {logit_acc:.4f}")
print(f"F1 score: {logit_f1:.4f}")

X_train_dense = X_train_tfidf.astype("float32").toarray()
X_test_dense = X_test_tfidf.astype("float32").toarray()
input_dim = X_train_dense.shape[1]

nn_model = keras.Sequential([
    keras.layers.Input(shape=(input_dim,)),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(1, activation="sigmoid"),
])

nn_model.compile(
    loss="binary_crossentropy",
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    metrics=["accuracy"],
)

print("\nTraining neural network")
history = nn_model.fit(
    X_train_dense,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2,
    verbose=2,
)

y_pred_proba = nn_model.predict(X_test_dense).ravel()
y_pred = (y_pred_proba >= 0.5).astype("int32")

nn_acc = accuracy_score(y_test, y_pred)
nn_f1 = f1_score(y_test, y_pred, average="binary")

print("\n=== Neural network model ===")
print(f"Accuracy: {nn_acc:.4f}")
print(f"F1 score: {nn_f1:.4f}")

```

```
print("\nClass mapping (numeric to label):")
for idx, name in enumerate(class_names):
    print(f"{idx} -> {name}")
```